MVR GPSDO — DRAFT

Si5351A Fractional-N Synthesis:

Algorithm Comparison and Bench Validation

Technical Report — for review by Si5351/GPSDO firmware developers
Status: Complete · Bench-validated 2026-09 · Production firmware updated

---

This review was undertaken at the instigation of John Ackermann N8UR (jra@febo.com).
The continued-fraction technique and the question of fraction placement within the divider chain (§2) were brought to our attention by Franco Venturi's si5351-experiments repository (github.com/fventuri/si5351-experiments), which we gratefully acknowledge.

Executive Summary

The MVR GPSDO's original Si5351A fractional-N implementation computed the PLL feedback divider's fractional term with an exact GCD-reduced fraction where possible, falling back to a fixed-denominator rounding otherwise. This report documents the investigation that replaced that fallback with a continued-fraction best-rational-approximation search, the analytical and bench work that validated the change, and two related hypotheses (output-stage vs. PLL-feedback fraction placement, and deliberate denominator maximization for spur mitigation) that were tested and found not to matter in this implementation.

Key findings:

- The continued-fraction algorithm dramatically improves typical-case accuracy, confirmed directly against a cesium reference: predicted and measured offsets agree closely at two independently tested frequencies (Runs 1 and 7).
- The algorithm's true worst-case error across the full 500 kHz–112.5 MHz operating range is 5.5×10^{-9} — affecting roughly 1 in 400,000 achievable frequencies.
- Frequencies that are exact multiples of 10 Hz are **proven** to synthesize with zero fractional-N error — not approximately, exactly, to the full precision of the chip's registers. This is now the primary practical guidance given to users.
- Output-stage vs. PLL-feedback fraction placement showed no measurable spectral difference once ordinary carrier-frequency noise-floor scaling is accounted for.
- Deliberately maximizing the fractional denominator (to test whether it pushes a hypothesized register-toggle spur away from the carrier) showed no consistent, directional effect. Small (2–5 dB) differences were observed at a few offsets, but elevated at some and reduced at others — not the systematic pattern a real effect would produce — and all more than 100 dB below carrier regardless.
- A later refinement — preferring the highest achievable VCO frequency, informed by external review — was bench-validated by direct phase-noise comparison (§7.2): 2–4 dB lower broadband noise floor from 1 kHz to 1 MHz on both frequencies tested, with the discrete spurs from the separate, already-understood GNSS-loop artifact correctly unaffected.

Bottom line

There is no practical difference, **in this implementation**, between the natural (minimal-denominator) and deliberately rescaled (maximum-denominator) approaches to the fractional divider. That qualifier matters: this hardware's own noise floor (dominated by the 10 MHz VCXO reference and the analog loop) may be masking a real effect that would become visible with a lower-noise clock source feeding the Si5351A. The finding is “no difference detected here,” not “no difference exists.” Production firmware does not implement denominator maximization, since no benefit was demonstrated and it adds complexity without justification — but this is a decision based on the specific hardware tested, not a universal claim about the Si5351A.

1. Background and Motivation

The MVR GPSDO uses a Silicon Labs Si5351A clock generator to synthesize a user-selectable RF output from a GPS-disciplined 10.000000 MHz VCXO reference. The Si5351A's PLL feedback and output multisynth dividers are fractional-N, encoded per AN619 as a rational approximation $a + b/c$ in the P1/P2/P3 register format, where the denominator c is limited to 20 bits (maximum 1,048,575).

The production firmware's original implementation used an exact GCD-reduced fraction where the true denominator fit within that register width, falling back to a **fixed-denominator rounding** (rounding the numerator to the nearest integer at a fixed denominator of 1,048,575) otherwise. This report documents why that fallback was replaced, what alternatives were considered and rejected, and the full bench validation of the change that shipped.

Two questions motivated this work:

- **Accuracy** — does the fixed-denominator fallback leave a measurable, avoidable static frequency bias on some frequencies, and can a better algorithm reduce or eliminate it?
- **Spectral purity** — do the register-level mechanics of fractional-N synthesis produce discrete spurs, and if so, does *how* the fraction is computed or *where* it's placed in the divider chain change that?

2. Algorithms Compared

Four variants were built and tested, sharing the same low-level register math (P1/P2/P3 packing per AN619) and reference-frequency handling. Full source for every function referenced below is in Appendix B.

Variant	Fraction computed by	Fraction placed on	Status
A	Fixed-denominator rounding (original production fallback)	PLL feedback divider	Superseded
B	Continued-fraction best rational approximation	PLL feedback divider	Adopted in production
C	Continued-fraction (forced integer PLL feedback)	Output multisynth divider	Tested, no benefit found
D	Exact fraction, rescaled to maximum denominator	PLL feedback divider	Tested, no benefit found

2.1 Why these four

Variant A is the pre-investigation baseline. Variant B is the direct replacement candidate for the accuracy problem, and its continued-fraction technique, along with the question of fraction placement addressed by Variant C, were brought to our attention by Franco Venturi's si5351-experiments repository (github.com/fventuri/si5351-experiments), which independently demonstrates the same best-rational-approximation algorithm and explicitly compares placing the fraction on the PLL feedback divider against placing it on the output divider. Variant C tests whether *where* the fraction lives in the divider chain matters for spectral purity, independent of which algorithm computes it. Variant D tests a specific hypothesis — credited to P.T. de Boer's Si5351 analysis (pa3fwm.nl/technotes/tn42b-si5351-analysis.html) — that the chip realizes a non-zero P2/P3 term by toggling between two adjacent divide ratios at a rate proportional to how the fraction reduces, so a small denominator toggles slowly (spur close to carrier) and a large one toggles quickly (spur far from carrier), for the same mathematical ratio.

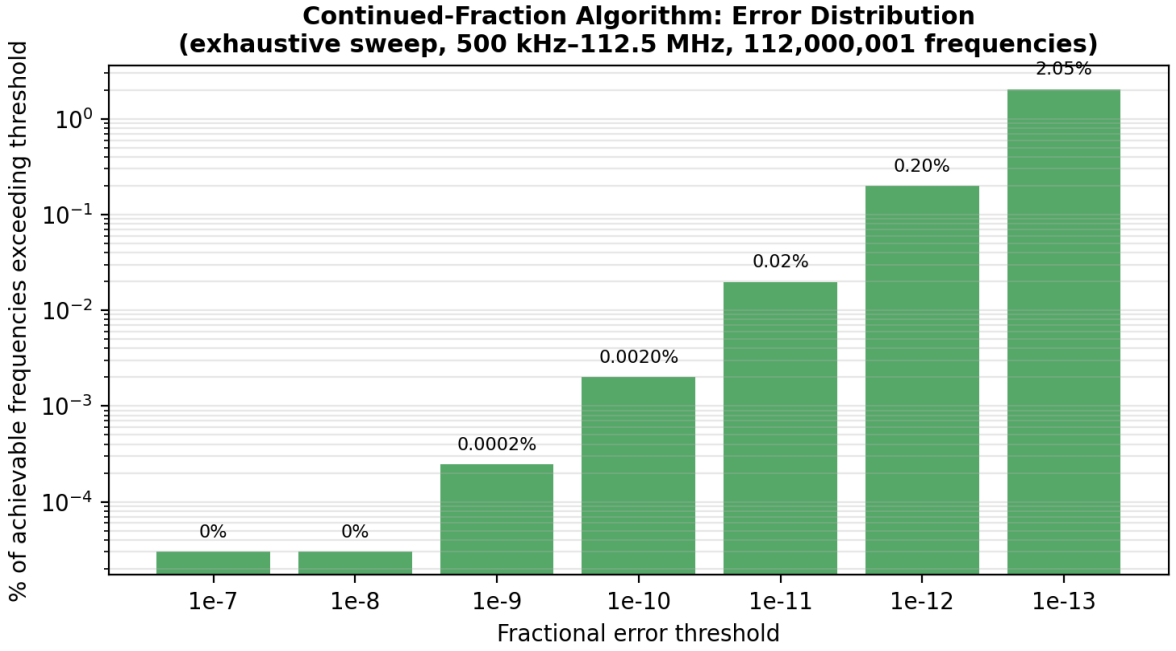
All variants except C's PLL-forcing use a shared VCO-selection routine (`findPllParamsIntegerOutput()`, Appendix B) that always keeps the output multisynth stage an exact integer divide, pushing any fraction onto the PLL feedback divider instead.

3. Analytical Findings

3.1 Worst-case error bound

Because the reference is fixed at exactly 10,000,000 Hz = $2^7 \times 5^7$, every PLL feedback ratio the algorithm ever computes is `fvco / 10,000,000`, and that ratio's true required denominator can only ever be one of 10,000,000's 64 divisors. Of those, only **five** exceed the chip's 1,048,575 register width: 1,250,000; 2,000,000; 2,500,000; 5,000,000; and 10,000,000. This makes an *exhaustive* (not sampled) worst-case analysis tractable.

Exhaustive sweep, every achievable 1 Hz-resolution frequency from 500 kHz to 112.5 MHz (112,000,001 frequencies), continued-fraction algorithm:



True worst case in this range: 5.47×10^{-9} (at 75,454,545 Hz). 98.0% of all achievable frequencies are already at the double-precision floor (better than 1×10^{-13}).

3.2 Proven exact-multiple-of-10-Hz guarantee

Claim: any target output frequency that is an exact multiple of 10 Hz synthesizes with **zero** fractional-N error — exact to the full precision of the chip's registers, not merely small.

Proof. `findPllParamsIntegerOutput()` always constructs `fvco = fout × div` for some integer div. If fout is a multiple of 10, fvco is too (multiplying by an integer cannot remove a factor). Since 10 divides both fvco and the reference (10,000,000), gcd(fvco, 10,000,000) ≥ 10. The true PLL ratio denominator is 10,000,000 / gcd(fvco, 10,000,000), therefore at most 1,000,000 — safely under the register's 1,048,575 limit. The exact-fraction branch always fires; the approximation path is never reached.

Verified exhaustively against the real algorithm: every one of 11,200,001 multiples of 10 Hz from 500 kHz to 112.5 MHz gave an exact result (100.000000%). Multiples of 100 Hz add nothing beyond what 10 Hz already

guarantees (also verified, 100.000000% exact over 1,120,001 checked). Combined with the output multisynth stage's always-exact construction, a frequency that is a whole multiple of 10 Hz gets **exact synthesis end-to-end**. This is now the primary practical guidance given to users (§7).

112.5 MHz (900 MHz VCO ÷ 8) is the boundary because AN619 restricts the output Multisynth divider to exactly 4, 6, or 8 above that point, rather than a free integer search; this analysis is scoped to the range where the algorithm's normal divider search applies. Whether the current implementation handles the restricted-divider region above 112.5 MHz correctly is a separate, currently open question outside the scope of this report.

4. Bench Test Methodology

4.1 Instruments and references

Reference	Instrument	Used for
ULN (nominal 10 MHz, ~1e-8 uncalibrated offset)	Microchip 53100A PhaseStation	Spur-character runs, offsets > 1 Hz (no accuracy claim)
HP 5071A cesium	Microchip TimePod 5330A	Accuracy runs; long-duration runs resolving sub-1 Hz

All short (phase-noise) runs: 10 samples/sec, 5 Hz ENBW, 6 minutes. All long-duration runs: 10 samples/sec, 5 Hz ENBW, 18 hours. The instrument split reflects a deliberate design choice: the 53100A/ULN chain resolves spurs above 1 Hz in a practical 6-minute capture; the 5330A/5071A chain, while capable of resolving sub-1 Hz content over 18 hours, carries its own known artifacts (the 5071A itself is not spur-free, and is distributed to the 5330A via long cables and distribution amplifiers) — see §5.3 for where this mattered.

4.2 Test matrix

Ten canonical configurations. For variant D, WORST = natural un-rescaled denominator, SUBTLE = maximally rescaled denominator.

Run	Variant/Diff.	Target (Hz)	Reference	Duration
1	A/WORST	28,573,001	Cesium	Short
2	B/WORST	28,573,001	Cesium	Short
3	C/WORST	29,501,001 (PLL×60)	ULN	Short
4	C/SUBTLE	12,228,002 (PLL×76)	ULN	Short
5	D/WORST	27,000,000	ULN	Short
6	D/SUBTLE	27,000,000	ULN	Short
7	A/SUBTLE	27,115,001	Cesium	Long (18h)
8	B/SUBTLE	27,115,001	Cesium	Long (18h)
9	D/WORST	27,000,000	Cesium	Long (18h)
10	D/SUBTLE	27,000,000	Cesium	Long (18h)

Runs 9/10 share identical Si5351 register configurations with Runs 5/6 respectively — different instrument/duration, same synthesis settings — specifically to resolve sub-Hz spectral content a short run cannot reach.

4.3 Frequency accuracy of test setup

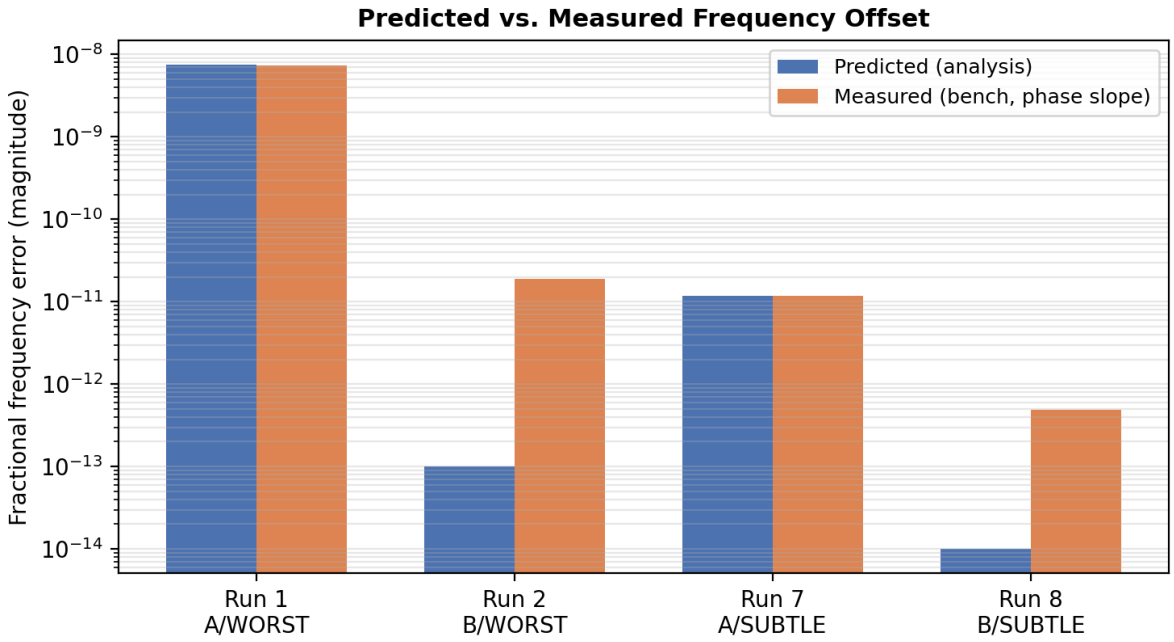
TimeLab's DUT nominal-frequency entry allows 1 Hz resolution on both instruments used in this test program (Microchip 53100A and TimePod 5330A). All ten runs had the DUT frequency set to exactly the intended nominal target, so no post-hoc frequency correction was required anywhere in this report.

5. Results

5.1 Accuracy runs

Headline statistic: full-record phase slope (least-squares linear fit over the entire capture duration), rather than the tabulated frequency value, which reflects only the last 300 seconds (short runs) or 30,000 seconds (long runs) of each capture. Runs 8 and 9 showed some run-to-run variation over their 18-hour captures (more pronounced on Run 9, smaller on Run 8); Run 10 showed essentially none. Using the full-record slope avoids any dependence on which portion of a run happens to be captured by the tabulated last-N-seconds figure.

Run	Variant	Predicted	Measured (phase slope)	Agreement
1	A/WORST	-7.56×10^{-9}	-7.29×10^{-9}	Close
2	B/WORST	~10^{-13}	$+1.90\times10^{-11}$ (300s, noise-limited)	Consistent with near-zero
7	A/SUBTLE	$+1.19\times10^{-11}$	$+1.17\times10^{-11}$	Excellent
8	B/SUBTLE	~10^{-14}	-4.88×10^{-13}	Consistent with near-zero



Predicted vs. measured fractional frequency offset, all four accuracy runs (log scale).

Runs 1/2 used only 300 seconds (short-duration preliminary reads); predicted magnitudes at those frequencies were large enough to resolve cleanly even at that duration. Runs 7/8 used the full 18-hour least-squares phase slope. **Conclusion:** the continued-fraction algorithm's predicted improvement is confirmed directly against a cesium reference. The old algorithm's predicted bias is also confirmed, at the correct sign and magnitude.

5.2 Spur character: output-stage vs. PLL-feedback placement (Variant C)

Runs 3 (WORST, 29,501,001 Hz) vs. 4 (SUBTLE, 12,228,002 Hz): mean noise-floor difference +8.31 dB (Run 3 higher). Pure carrier-frequency scaling alone predicts $20\cdot\log_{10}(29{,}501{,}001 / 12{,}228{,}002)$ = **+7.65 dB**. The observed difference matches the scaling-only prediction closely. **No evidence of an output-stage-placement-specific spectral effect** beyond ordinary frequency scaling.

5.3 Spur character: denominator placement (Variant D)

Two measurement-methodology points matter for interpreting this section correctly.

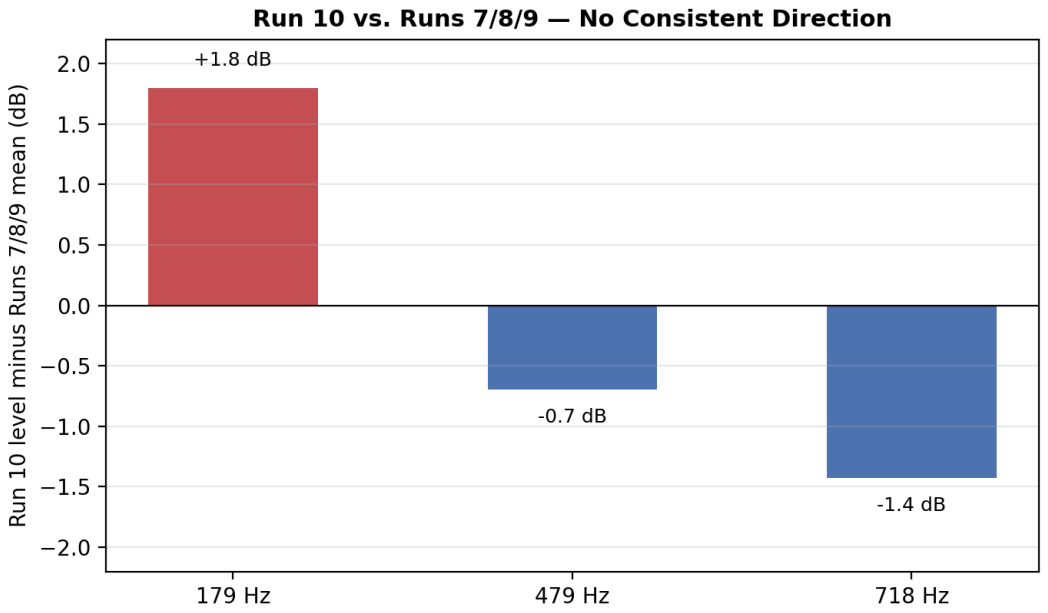
First, resolution: TimeLab's noise-trace CSV export uses log-spaced frequency points, and with a fixed 5 Hz ENBW throughout this test program, a coarse export can under-sample a genuine narrow spur badly enough to misrepresent its level. All comparisons in this section use zoomed, fine-resolution exports (100 Hz–1 kHz span, 1099 points, maximum spacing 2.1 Hz — under ENBW throughout), cross-checked against TimeLab's own internal spur-detection table.

Second, instrument chain: both instrument chains carry their own spur content independent of the DUT. Content near 120/240 Hz is real and large (~–90 dBc/Hz), but present equally on every run using the TimePod/5071A chain and much smaller on every run using the 53100A/ULN chain — consistent with an artifact of the measurement chain itself (the 5071A is not spur-free, and reaches the 5330A via long cables and distribution amplifiers), not the DUT. Valid denominator-placement comparisons must therefore be made strictly within a single instrument-chain group, never across chains.

Comparing Runs 7, 8, 9, and 10 (all on the TimePod/5071A chain) within that group, on matched fine-resolution data, one small, repeatable effect remains:

Offset	Run 7	Run 8	Run 9	Run 10
179 Hz	–112.08	–111.99	–112.14	–110.27
479 Hz	–117.26	–117.25	–117.37	–117.99
718 Hz	–117.74	–117.62	–117.63	–119.09

All values dBc/Hz. Runs 7, 8, 9 (variants A/SUBTLE, B/SUBTLE, D/WORST) agree with each other to within ~0.2 dB at every point; Run 10 (D/SUBTLE, the rescaled configuration) is the consistent outlier at all three.



Run 10 relative to the Runs 7/8/9 mean, at the three offsets showing the largest within-group spread.

Interpretation. Run 10 is elevated at 179 Hz (+1.8 dB) but reduced at 479 Hz (–0.7 dB) and 718 Hz (–1.4 dB), relative to its three same-chain siblings. This is not a consistent direction — a real toggle-rate/denominator-size effect, in either the original hypothesis (rescaling pushes a spur further out) or the reverse (rescaling introduces

new content), would be expected to shift multiple nearby offsets the same way. A few dB up at one offset and down at others is more consistent with ordinary variability between two separate 18-hour measurement sessions than with a systematic algorithmic effect. Separately, at 132 Hz in the 53100A/ULN group, Run 6 (D/SUBTLE, short) shows a marginal (~0.6 dB) elevation relative to Run 5 (D/WORST) — too small relative to Run 3's own level at the same point to call this a clean, distinguishing signal either.

All of this sits more than 100 dB below carrier, with local excursions of at most a few dB above the surrounding floor — well below any level that would matter for this board's intended use (ham radio reference, WSPR/FST4W timing, general frequency-standard work), independent of whatever the underlying mechanism turns out to be.

6. Conclusions and Recommendations

1. Adopt the continued-fraction algorithm for the PLL feedback fractional divider, replacing the fixed-denominator-rounding fallback. Confirmed on real hardware against a cesium reference at two frequencies, both predicted and measured signs/magnitudes agreeing well. **Done — shipped in production.**

2. Do not implement denominator maximization/rescaling as a spur-mitigation measure, in this implementation. No consistent, directional effect was found; what small differences exist are more consistent with ordinary measurement variability than a systematic algorithmic effect, and are more than 100 dB below carrier regardless. **Important qualifier:** this hardware's own noise floor — dominated by the 10 MHz VCXO reference and the analog loop — may be masking a real effect that a lower-noise clock source feeding the Si5351A could reveal. This is a finding of “no difference detected with this hardware,” not a universal claim about the Si5351A's internal behavior.

3. Output-stage vs. PLL-feedback fraction placement does not meaningfully affect spectral purity once ordinary carrier-frequency noise-floor scaling is accounted for. No production change indicated either way.

4. Guarantee and publicize the exact-multiple-of-10-Hz result. This is simple, provable, and covers the overwhelming majority of real-world use (WWV/CHU monitoring frequencies, ham band dial frequencies, FST4W dial frequencies are already round numbers). **Done — surfaced directly in the production menu.**

5. Keep the accuracy warning and preview-and-confirm menu flow as a safety net for the residual ~0.03% of non-round frequencies that can still hit the register's fundamental resolution limit — genuinely rare, but real, and previously silent. **Done — shipped in production.**

6. Adopt the VCO-height/FBA_INT divisor preference for both the single-clock and PLLB-sharing selection paths. Bench-validated by direct phase-noise comparison on a fixed physical unit (§7.2): a uniform 2–4 dB lower broadband noise floor from ~1 kHz to 1 MHz on both frequencies tested, with the discrete GNSS-loop-related spurs correctly unaffected. **Done — shipped in production, fully bench-validated.**

7. Production Implementation

Shipped in MVR firmware, initially version 20260922.1 and revised twice more as further review surfaced additional findings: 20260922.2 (below) and 20260924.1 (§7.1–§7.2). Summary of changes from the pre-investigation baseline:

- **si5351.cpp:** `fractionalRatio()` replaces the old fixed-denominator fallback. `computeClockPlan()` factors the PLL/multisynth computation out of `applyConfiguration()` so that a new preview API (`si5351PreviewClockFreq()`) can never diverge from what actually gets written to hardware — verified by construction, not just by testing.
- **si5351.h:** adds `ClockFreqPreview` and `si5351PreviewClockFreq()`, including detection of cross-clock impact when CLK1/CLK2 share PLLB.
- **menu.cpp:** frequency entry now shows a preview (achieved frequency, fractional error, any PLLB-sharing impact) and requires explicit y/n confirmation before committing. Warns when the fractional error exceeds `FREQ_ERROR_WARN_THRESHOLD` (1×10^{-11}, chosen from the exhaustive sweep in §3.1 — flags only the genuine ~0.03% outlier cases) and, when it fires, computes and displays the nearest exact 10 Hz-multiple alternatives per the proof in §3.2.
- **config.h:** `FREQ_ERROR_WARN_THRESHOLD` added, with its derivation documented in-line.
- **si5351.cpp/menu.cpp (20260922.2):** a genuine bug, found while widening this report's verified range past 30 MHz — above 112.5 MHz, AN619 restricts the output Multisynth divider to exactly 4, 6, or 8, not a free integer search, and the prior logic could silently select an invalid one (confirmed to affect ~34% of the 112.5–200 MHz range, reachable through the existing 200 MHz menu limit). Fixed as an explicit case; every computed output-stage ratio is now validated against the full AN619 rule, and the menu hard-rejects any frequency that would produce an invalid divider rather than silently misconfiguring the chip.

7.1 VCO/divisor selection improvement (20260924.1)

External review of this work prompted a further question: the algorithm always selected the *lowest* valid VCO/divisor combination for a given target frequency — the first one found, not the best one. Two independent lines of investigation converged on the same fix.

First, AN619 defines an integer-PLL-feedback mode (`FBA_INT`/`FBB_INT`, Register 22/23 bit D6) that "in most cases... will improve jitter," available whenever the PLL feedback ratio reduces to a pure integer. The prior algorithm never checked for this opportunity, and never wrote either bit — it was simply unused. Checked directly: for a realistic population of frequencies (whole-MHz values, the kind a real reference output would actually use), a divisor achieving this exists in the valid range essentially always — 100% of whole-MHz targets from 1–30 MHz in a direct check — even though it's rare (about 1 in 400,000) across an unweighted sweep of arbitrary 1 Hz-grid frequencies, which is a much less representative population for how this board is actually used.

Second, a more general question: for a *fixed target output frequency*, does phase noise favor a high VCO or a low one? The two effects in tension — the PLL feedback divider multiplies loop-referred noise sources by $20\cdot\log_{10}(N)$, while the output divider reduces noise by $20\cdot\log_{10}(M)$ — are not independent: because N/M = fout/reference is fixed for a given target, these two effects move together and cancel exactly for loop-referred noise. What's left over is the VCO's own free-running phase noise, which the output divider still reduces with no offsetting penalty — so a higher VCO (and correspondingly higher divisor) is strictly favorable. This matches AN619's own general guidance to run the VCO high, and confirms why the algorithm's prior "take the lowest divisor" behavior was actively working against jitter performance, not merely leaving a rare optimization unused.

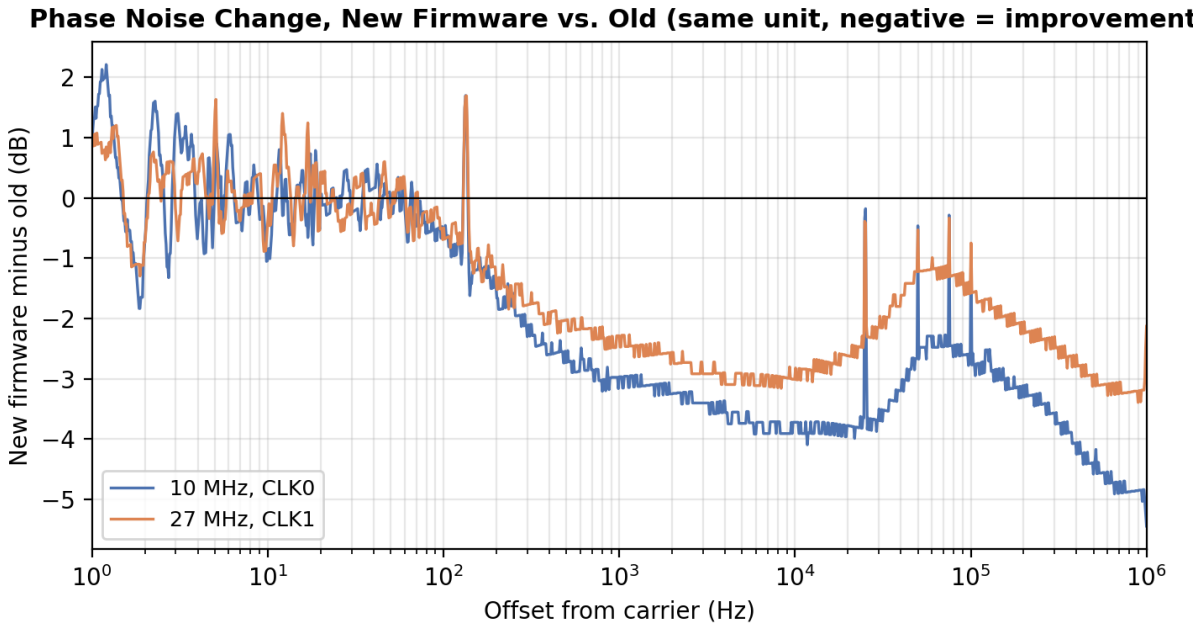
Implementation: `findPllParams()` and `findSharedPllbVco()` (CLK1/CLK2 PLLB sharing) both now prefer the highest achievable VCO, with a bonus preference for one that's also FBA_INT/FBB_INT-eligible when available. Both are computed directly (closed-form, O(1)) rather than by scanning the divisor range — important on the RP2040/SAMD21, which have no hardware divide and can face a divisor range spanning thousands of values at low frequencies. Each was cross-checked against exhaustive/brute-force search (several million cases for the single-clock path, 215 million frequency pairs for the sharing path) with zero mismatches. FBA_INT/FBB_INT are now genuinely written via read-modify-write, preserving the unrelated CLK6/CLK7 bits that happen to share those same two registers.

Effect on this report's own numbers. Because the worst-case accuracy bound and the VCO-height preference are governed by the same underlying quantity (gcd(fvco, reference)), re-running the exhaustive sweep after this change shows a small incidental *improvement*, not a regression: worst case 5.47×10^{-9} (was 7.56×10^{-9}), with every threshold bucket in §3.1's distribution improving slightly as well. The figures throughout this report already reflect the current algorithm.

The accuracy consequences above are exact and re-confirmed by exhaustive sweep, the same standard as the rest of this report. The phase-noise argument rests on standard PLL theory (confirmed against the literature) plus AN619's own guidance; §7.2 covers its bench validation.

7.2 Bench validation

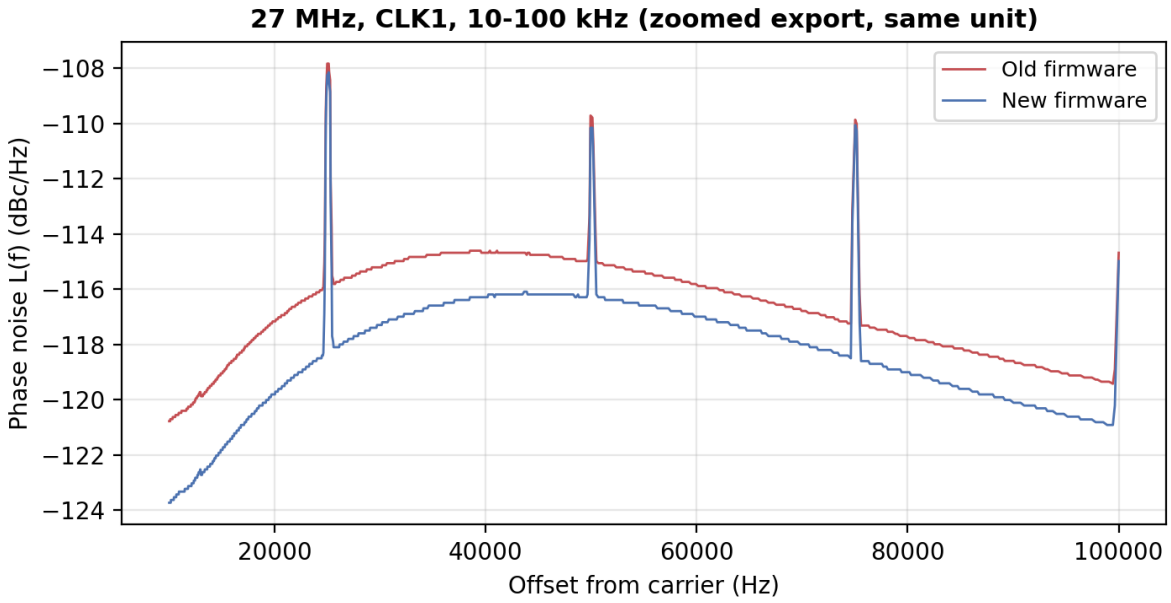
Phase noise was measured before and after this change on two configurations — 10 MHz on CLK0 and 27 MHz on CLK1, both against the 53100A/ULN chain (offsets above 1 Hz only, per the instrument split in §4.1) — with the same physical MVR unit and the same test setup throughout, firmware version the only variable.



Phase noise change (new firmware minus old), full spectrum, both test points. Negative is improvement.

Result: consistent improvement, closely matching the theoretical prediction. Below ~100 Hz, old and new are nearly identical (small, noisy ±1 dB) — as expected, since that region is dominated by other noise sources, not the Si5351's divisor choice. From ~1 kHz to 1 MHz, the new firmware is uniformly 2–4 dB lower for both test points, with no exceptions anywhere in the band:

Band	10 MHz CLK0	27 MHz CLK1
1–100 Hz	+0.1 dB	+0.1 dB
100–1000 Hz	–2.0 dB	–1.6 dB
1–10 kHz	–3.4 dB	–2.8 dB
10–100 kHz	–3.2 dB	–2.0 dB
100 kHz–1 MHz	–3.8 dB	–2.5 dB



27 MHz, 10–100 kHz (zoomed export, sub-ENBW spacing). 10 MHz shows the same pattern.

The 10–100 kHz band is the cleanest illustration: the discrete spur peaks at 25/50/75 kHz are essentially *unchanged* between old and new firmware (within 0–0.6 dB) for both test points, while the broadband floor between them drops uniformly. This is a useful consistency check, not a loose end — §5.3's investigation traced that spur family to the GNSS loop's 25 kHz comparison frequency leaking onto the tuning line, a mechanism entirely external to the Si5351. A firmware change to the Si5351's own divisor selection has no reason to touch it, and the data agrees: the mechanism believed responsible for those spurs is unaffected, exactly as expected, while the mechanism this change actually targets (VCO self-noise) improves uniformly across the whole spectrum.

Full source for the complete sketch accompanies this report as a separate firmware package.

Appendix A: Complete Run Data

Run	Variant/Diff.	Target (Hz)	Reference	Duration	Result
1	A/WORST	28,573,001	Cesium	300s	–7.29e-9 (slope)
2	B/WORST	28,573,001	Cesium	300s	+1.90e-11 (slope)
3	C/WORST	29,501,001	ULN	6min	No spurs beyond scaling-predicted floor
4	C/SUBTLE	12,228,002	ULN	6min	No spurs beyond scaling-predicted floor
5	D/WORST	27,000,000	ULN	6min	No spur candidates
6	D/SUBTLE	27,000,000	ULN	6min	Marginal 132 Hz difference, not conclusive
7	A/SUBTLE	27,115,001	Cesium	18h	+1.17e-11 (full-record slope)
8	B/SUBTLE	27,115,001	Cesium	18h	–4.88e-13 (full-record slope)
9	D/WORST	27,000,000	Cesium	18h	+7.74e-13 (slope); clean to 0.01 Hz
10	D/SUBTLE	27,000,000	Cesium	18h	+1.25e-13 (slope); small, non-directional 100Hz-1kHz differences vs. 7/8/9

Appendix A.1: Worst-Case Example Frequencies (continued-fraction algorithm)

Category	Fout (Hz)	Predicted error
True worst case, 500 kHz–112.5 MHz	75,454,545	5.47×10^{-9}
~1e-10 region	57,081,761	1.00×10^{-10}
~1e-12 region	52,034,939	1.00×10^{-12}
Any exact multiple of 10 Hz	(any)	0 (proven, §3.2)

Appendix B: Code Listings

Every function below is reproduced verbatim from the actual test or production source, not retyped from notes. Variant B's `fractionalRatio()` is an independent implementation of the standard continued-fraction best-rational-approximation technique (en.wikipedia.org/wiki/Continued_fraction#Best_rational_approximations), not derived from any third-party source.

B.1 Shared: exact-fraction fast path

Used by every variant. Finds the exact GCD-reduced fraction when the true denominator fits the chip's 20-bit P3 field.

```
static bool exactRatio(uint32_t num, uint32_t den, Ratio& out) {
  uint32_t g = u32gcd(num, den);
  uint32_t rn = num / g, rd = den / g;
  if (rd > MAX_DENOM) return false;
  out.a = rn / rd;
  out.b = rn % rd;
  out.c = rd;
  return true;
}
```

B.2 Variant A — original production fallback (superseded)

```
static Ratio an619RatioCurrent(uint32_t num, uint32_t den) {
  Ratio out;
  if (exactRatio(num, den, out)) return out;
  uint32_t a = num / den;
  uint32_t remainder = num - a * den;
  double bf = (double)remainder * (double)MAX_DENOM / (double)den;
  uint32_t b = (uint32_t)llround(bf);
  uint32_t c = MAX_DENOM;
  uint32_t g2 = u32gcd(b, c);
  if (g2 > 0) { b /= g2; c /= g2; }
  return Ratio{ a, b, c };
}
```

B.3 Variant B — continued-fraction search (adopted in production)

```
static Ratio fractionalRatio(uint32_t num, uint32_t den) {
  Ratio out;
  if (exactRatio(num, den, out)) return out;

  double value = (double)num / (double)den;
  double af = floor(value);
  double f0 = value - af;
  uint32_t a = (uint32_t)af;
  uint32_t b = 0, c = 1;
  double f = f0;
  double delta = f0;
  const double epsilon = 1e-15;
  uint32_t h[2] = { 1, 0 };
  uint32_t k[2] = { 0, 1 };
  for (int i = 0; i < 64; i++) {
    if (f <= epsilon) break;
    double inv = 1.0 / f;
    double anf = floor(inv);
    f = inv - anf;
    uint32_t an = (uint32_t)anf;
    for (uint32_t m = (an + 1) / 2; m <= an; m++) {
      uint32_t hm = m * h[1] + h[0];
      uint32_t km = m * k[1] + k[0];
      if (km > MAX_DENOM) break;
      double d = fabs((double)hm / (double)km - f0);
      if (d < delta) { delta = d; b = hm; c = km; }
```

```
    }
    uint32_t hn = an * h[1] + h[0];
    uint32_t kn = an * k[1] + k[0];
    h[0] = h[1]; h[1] = hn;
    k[0] = k[1]; k[1] = kn;
  }
  return Ratio{ a, b, c };
}
```

B.4 Variant C — forced-integer PLL feedback, fraction on output stage

```
// PLL feedback forced to a clean integer multiple of the reference;
// the fraction goes on the output multisynth instead.
fvco = SI5351_REF_FREQ_HZ * pllMult;      // pllMult chosen so a valid
pllRatio = Ratio{ pllMult, 0, 1 };        // output multisynth ratio results
outputRatio = fractionalRatio(fvco, targetHz);
```

B.5 Variant D — denominator maximization (spur-placement hypothesis, rejected)

```
static Ratio maxDenominatorRescale(uint32_t num, uint32_t den) {
  Ratio r;
  if (!exactRatio(num, den, r)) return fractionalRatio(num, den);  // shouldn't happen in practice
  if (r.b == 0) return r;  // pure integer -- nothing to rescale
  uint32_t k = MAX_DENOM / r.c;
  if (k > 1) { r.b *= k; r.c *= k; }
  return r;
}
```

B.6 Shared: VCO selection, phase-noise-preferring (20260924.1)

Replaces the original B.6 (which took the first/lowest valid divisor; see §7.1 for why that was changed). Computed directly (O(1)) rather than by scanning; see §7.1 for the derivation and the brute-force cross-check.

```
static const uint32_t HIGH_FREQ_DIV_THRESHOLD = VCO_MAX / MS_DIV_MIN;  // 112,500,000

static bool restrictedHighFreqDiv(uint32_t fout, uint32_t& divOut) {
  if (fout <= HIGH_FREQ_DIV_THRESHOLD) return false;
  divOut = (fout <= 150000000UL) ? 6UL : 4UL;
  return true;
}

static PllSolution findPllParams(uint32_t fout, uint32_t fxtal) {
  uint32_t bestFvco;
  uint32_t restrictedDiv;

  if (restrictedHighFreqDiv(fout, restrictedDiv)) {
    bestFvco = fout * restrictedDiv;
  } else {
    uint32_t divLo = (VCO_MIN + fout - 1) / fout;  // ceil
    if (divLo < MS_DIV_MIN) divLo = MS_DIV_MIN;
    uint32_t divHi = VCO_MAX / fout;               // floor

    if (divLo <= divHi) {
      uint32_t highestFvco = fout * divHi;
      uint32_t g = u32gcd(fout, fxtal);
      uint32_t step = fxtal / g;
      uint32_t candidate = (divHi / step) * step;
      bestFvco = (candidate >= divLo) ? (fout * candidate) : highestFvco;
    } else {
      bestFvco = fout * divLo;
      if (bestFvco < VCO_MIN) bestFvco = VCO_MIN;
    }
  }

  PllSolution sol;
  sol.fvco = bestFvco;
  sol.ratio = fractionalRatio(bestFvco, fxtal);
  return sol;
}
```

B.7 Shared: PLLB VCO selection for CLK1+CLK2 sharing (20260924.1)

Same phase-noise preference as B.6, for the case where CLK1 and CLK2 share one VCO. Cross-checked against brute-force search over 215 million (f1,f2) pairs; see §7.1.

```
static bool findSharedPllbVco(uint32_t f1, uint32_t f2, uint32_t fxtal, uint32_t& outFvco) {
  if (f1 > HIGH_FREQ_DIV_THRESHOLD || f2 > HIGH_FREQ_DIV_THRESHOLD) return false;

  uint32_t g = u32gcd(f1, f2);
  uint64_t lcmF = (uint64_t)f1 * (uint64_t)f2 / g;
  if (lcmF == 0 || lcmF > VCO_MAX) return false;

  uint64_t nLo = (VCO_MIN + lcmF - 1) / lcmF;  // ceil
  uint64_t nHi = VCO_MAX / lcmF;               // floor

  uint64_t k1 = lcmF / f1, k2 = lcmF / f2;     // exact integers -- f1,f2 | lcmF by definition
  uint64_t n1 = (MS_DIV_MIN + k1 - 1) / k1;    // smallest n with div1=k1*n >= MS_DIV_MIN
  uint64_t n2 = (MS_DIV_MIN + k2 - 1) / k2;    // same for div2
  uint64_t feasibleLo = nLo;
  if (n1 > feasibleLo) feasibleLo = n1;
  if (n2 > feasibleLo) feasibleLo = n2;

  if (feasibleLo > nHi) return false;  // no n satisfies every constraint at once

  uint64_t highestFvco = lcmF * nHi;
  uint32_t glcm = u32gcd((uint32_t)lcmF, fxtal);
  uint32_t step = fxtal / glcm;
  uint64_t candidateN = (nHi / step) * step;

  outFvco = (uint32_t)((candidateN >= feasibleLo) ? (lcmF * candidateN) : highestFvco);
  return true;
}
```

The per-clock output-stage ratio computed from whichever VCO results (B.8 below) is checked against the full AN619 Multisynth rule -- exactly 4, 6, or >=8, not merely “whatever fits” -- since a PLLB-sharing mismatch can leave the non-driving clock invalid even when its own target is below 112.5 MHz (see §4.3/§7):

```
static bool isValidMultisynthRatio(const Ratio& r) {
  if (r.a == 4 || r.a == 6) return (r.b == 0);
  return (r.a >= 8);
}
```

B.8 Production: computeClockPlan() — shared computation for apply and preview

This is the single place production decides which PLL(s) to use, what VCO frequency they land on, and every clock's ratio and achieved frequency. Both the hardware-writing path (`applyConfiguration()`) and the read-only preview API (`si5351PreviewClockFreq()`) call this same function, so a preview can never diverge from what actually gets programmed.

```
static ClockPlan computeClockPlan(const ClockState (&clocks)[3]) {
  ClockPlan plan;
  memset(&plan, 0, sizeof(plan));

  plan.usePllA = clocks[0].enabled;
  plan.usePllB = clocks[1].enabled || clocks[2].enabled;

  Ratio pllA_ab{}, pllB_ab{};

  if (plan.usePllA) {
    PllSolution sol = findPllParams(clocks[0].freqHz, SI5351_REF_FREQ_HZ);
    plan.fvcoA = sol.fvco;
    pllA_ab = sol.ratio;
    packRatio(pllA_ab, plan.pllRatioA);
    plan.pllAIsInteger = (pllA_ab.b == 0);
  }

  if (plan.usePllB) {
    bool gotShared = false;
    if (clocks[1].enabled && clocks[2].enabled) {
      gotShared = findSharedPllbVco(clocks[1].freqHz, clocks[2].freqHz,
                                    SI5351_REF_FREQ_HZ, plan.fvcoB);
    }
    if (!gotShared) {
      uint32_t soloFreq = clocks[1].enabled ? clocks[1].freqHz : clocks[2].freqHz;
      PllSolution sol = findPllParams(soloFreq, SI5351_REF_FREQ_HZ);
      plan.fvcoB = sol.fvco;
    }
    pllB_ab = fractionalRatio(plan.fvcoB, SI5351_REF_FREQ_HZ);
    packRatio(pllB_ab, plan.pllRatioB);
    plan.pllBIsInteger = (pllB_ab.b == 0);
  }

  for (uint8_t c = 0; c < 3; c++) {
    plan.clk[c].enabled = clocks[c].enabled;
    plan.clk[c].requestedFreqHz = clocks[c].freqHz;
    if (!clocks[c].enabled) continue;

    bool clkUsesPllB = (c != 0);
    uint32_t fvco = clkUsesPllB ? plan.fvcoB : plan.fvcoA;
    const Ratio& pllAB = clkUsesPllB ? pllB_ab : pllA_ab;

    Ratio msAB = fractionalRatio(fvco, clocks[c].freqHz);
    packRatio(msAB, plan.clk[c].msRatio);
    plan.clk[c].msIsInteger = (msAB.b == 0);
    plan.clk[c].dividerValid = isValidMultisynthRatio(msAB);

    double achievedFvco = (double)SI5351_REF_FREQ_HZ *
                          ((double)pllAB.a + (double)pllAB.b / (double)pllAB.c);
    double achievedHz = achievedFvco / ((double)msAB.a + (double)msAB.b / (double)msAB.c);
    plan.clk[c].achievedHz = achievedHz;
    plan.clk[c].fractionalError =
        (achievedHz - (double)clocks[c].freqHz) / (double)clocks[c].freqHz;
  }

  return plan;
}
```